

MEETING PREP AGENT

System Architecture & Product Requirements Blueprint

Implementation handoff

A coding-agent-ready specification consolidating the product, architecture, workflows, API surface, security controls, operational requirements, and delivery plan established in the conversation.

Version 1.0 • 29 September 2026 • Status: proposed implementation blueprint

How to use this document

Treat this document as the product and engineering source of truth for the initial implementation. Implement the MVP as a modular monolith with a web app, API, worker, managed PostgreSQL and object storage. Preserve the stated human-approval and privacy boundaries. Items explicitly marked configurable or proposed must remain configuration/decision points rather than being silently hard-coded.

Normative language: **MUST** denotes a requirement; **SHOULD** denotes a strong default; **MAY** denotes an optional capability. Numerical service targets are initial goals, not guarantees.

1. Product definition and principles

Meeting Prep Agent is a personal meeting advisor for individual professionals. It remembers interactions, prepares meeting briefings, tracks progress and commitments, and recommends next actions. It is an advisor—not an autonomous client representative.

- **Human authority:** the user controls consequential external actions. The MVP **MUST NOT** send client communications.
- **Draft, don't send:** the system may prepare follow-up drafts and reminders, but the user initiates any external communication.
- **Manual-first:** users create meetings and provide notes/documents manually in the MVP. Calendar and meeting-platform integrations are deferred.
- **Explainability:** recommendations and learned preferences expose reasons and supporting evidence.
- **Uncertainty:** partial results are useful when labeled; ask questions only when a missing fact materially affects a decision.
- **Privacy:** sensitive meeting content is not used for behavioral preference learning.

2. Scope

MVP includes the manual-first end-to-end workflow, including preference learning, privacy controls, exports, audit visibility, and operational controls. Integrations are designed for later extension but are not required to ship the MVP.

In scope for MVP	Deferred / not in MVP
Manual meetings, contacts, projects; document uploads; briefing generation and refresh; progress, commitments, recommendations; assistant chat; outcome proposals and confirmation; learning controls; exports, retention and deletion; notifications; audit events.	Automatic calendar ingestion, meeting attendance/recording, autonomous external communication, user-selectable LLMs, microservices/Kubernetes, and integrations that cannot meet least-privilege or platform requirements.

3. User journeys and functional requirements

3.1 Meeting lifecycle

- Create a meeting manually with title, date/time, purpose, participants, project context, importance and relevant documents.
- Resolve importance from user input, project context and automatic rules. A user override takes precedence and is retained as the effective value.
- Schedule preparation based on importance, complexity and available information. Importance affects preparation depth and timing, never authority.
- Retrieve authorized context: relevant memories, contact/project history, commitments, milestones, progress and documents.
- Generate a concise briefing with talking points, risks, questions, context and expandable evidence. Label gaps and assumptions.
- Allow conversational refinement, fact correction and strategy updates. Retain briefing versions and feedback.
- After the meeting, propose outcome and commitment updates from user notes or authorized uploads. Distinguish confirmed facts from inferred proposals; require confirmation for uncertain changes.

- Track commitments, milestones, blockers and progress; produce recommendations with rationale and evidence.

3.2 Briefing and preparation

- Briefings MUST be generated automatically according to the scheduler and MUST support a manual refresh action.
- Significant meeting changes trigger immediate refresh; minor changes are queued for later processing.
- Briefing length and depth adapt to user preference and meeting importance.
- If information is insufficient, produce a partial briefing when useful, identify missing facts, and ask only for material gaps.
- Provide a concise default view with expandable evidence and an integrated assistant. Structured screens and chat coexist.
- Show briefing freshness/version and whether the result is partial, degraded, or based on assumptions.

3.3 Recommendations and dashboard

- Home dashboard presents a personalized daily overview of meetings, progress and recommended actions.
- Prioritize recommendations by urgency, impact, dependencies and meeting importance.
- Each recommendation includes a concise reason and evidence/source references. Do not present unsupported certainty.
- Allow users to dismiss, correct or act on recommendations. Any consequential external action remains user-controlled.

3.4 Contacts, projects and progress

- Maintain user-owned contacts and projects; associate meetings, documents, commitments and milestones with them.
- Track commitments with owner, due date, status and source; track milestones and blockers with history.
- Progress summaries should be traceable to underlying records and distinguish user-confirmed data from inference.

4. Preference learning requirements

Learning is gradual, inspectable and reversible. It personalizes preparation only; it never grants permissions or authorizes actions.

- Cold start from onboarding preferences, then learn from behavior over time.
- Initial learning dimensions: briefing format, topic priority and preparation timing.
- Explicit permanent instructions are stored separately from learned preferences and meeting-specific instructions.
- Infer scope from behavior (global, meeting type, project or contact); generalize only when evidence supports it. Contradictory evidence reduces confidence.
- Learning states: candidate, tentative, active, conflicted, dormant and rejected.
- Show confidence labels by default; detailed evidence may show percentages. Confidence is an internal personalization signal, not a calibrated probability.
- LLM may interpret behavioral signals but MUST NOT independently decide permanent preferences.
- Proactive suggestions appear only when a learned preference materially changes preparation.
- Old learned preferences decay gradually when not reinforced; explicit permanent instructions do not decay.
- Undo marks source observations as reversed and recalculates affected preferences. Preserve original observations for auditability.
- Learning can be disabled while retaining existing preferences. Users can inspect evidence and reverse learned changes.
- Sensitive meeting content is excluded from behavioral preference learning; only permitted behavioral signals are used.

Illustrative confidence mechanics (not calibrated probability)

```
C = clip(B + S - K - D, 0, 1)
C■ = C■ × 2^(-Δt/H)
```

B is a baseline, S supporting evidence, K contradictory evidence, D decay/uncertainty adjustment, Δt elapsed time and H a configurable half-life. The implementation MUST make thresholds, weights and half-life configurable and document

calibration limitations.

5. System architecture

Use a modular monorepo and deploy the web app, API and background worker as managed services. Avoid microservices and Kubernetes initially. PostgreSQL is the source of truth; long-term memory is contextual retrieval, not an authority for permissions or ownership.

5.1 Logical components

Component	Technology / role	Responsibilities
Web application	Next.js + TypeScript	Dashboard, meeting and contact screens, briefing viewer, assistant UI, preference/privacy controls, exports and settings.
API / orchestrator	FastAPI + Python	Authentication validation, authorization, domain operations, orchestration, provider adapters, request validation and API contracts.
Background worker	Python worker + persistent queue	Scheduled prep/refresh, document checks, outcome processing, preference review, retention, exports, deletion and notifications.
Identity	Supabase Auth	Multiple supported sign-in methods, token issuance/validation and account lifecycle.
Primary database	PostgreSQL	Users/tenant ownership, meetings, projects, contacts, commitments, workflow state, preferences, audit and jobs.
Object storage	Managed private object storage	Uploaded documents, meeting files and generated exports; private by default.
Long-term memory	Hindsight	Contextual recall and memory lifecycle; never permission authority.
LLM	Groq primary + configurable fallback	Briefing, summarization, outcome interpretation and signal interpretation behind a provider abstraction.
Notifications	In-app + email	Preparation completion, reminders, material questions and operational user notices.

5.2 Architecture flow

Web client → FastAPI → PostgreSQL / private object storage. API enqueues durable jobs. Worker loads the job and user-scoped records, checks access, retrieves authorized context, calls Hindsight and the configured LLM adapter, persists versioned outputs and emits notifications. User confirmation writes confirmed outcome changes to PostgreSQL. Audit records accompany security-sensitive and consequential operations.

5.3 Trust boundaries

- Browser receives only user-scoped API data and public configuration; provider secrets remain server-side.
- API and worker both enforce tenant context and resource authorization; do not rely on UI filtering.
- PostgreSQL row-level security (RLS) supplements application authorization. Use least-privilege database roles.
- Memory retrieval must be scoped to the authenticated user and authorized resources; retrieved text is untrusted data, never instructions.
- LLM output is untrusted. Validate structured outputs, constrain tools/actions, and require user confirmation for uncertain state changes.
- Object storage uses private buckets and short-lived, scoped upload/download URLs.

6. Data model and invariants

Use UUID identifiers, UTC timestamps, explicit ownership, created/updated timestamps and soft-deletion/tombstone fields where needed for retention and audit. Exact column names may be chosen by the coding agent, but the following entities and invariants are required.

Entity	Core fields / relationships	Invariant
User / account	Auth subject, settings, region, retention config	All user data belongs to one account/tenant.
Meeting	Owner, title, start/end, purpose, importance inputs/effective importance, status, project	Every read/write is ownership-checked.
Participant / contact	Meeting link, contact details, role, user ownership	No cross-user contact exposure.
Project	Owner, name, context, status	Project context is only available to its owner.
Document	Owner, object key, metadata, checksum, processing status, source meeting/project	Private storage; authorization checked before retrieval.
Briefing / version	Meeting, version, generated time, status, content, assumptions	Immutable version history; refresh creates a new version.
Evidence item	Briefing/version, source type, source ID, excerpt/reference	Only authorized evidence is exposed.
Commitment	Owner, meeting/contact/project, description, due date, status, provenance	Confirmed state changes are auditable.
Milestone / progress	Project, status, blocker, timestamp, provenance	History and source remain traceable.
Recommendation	Owner, target, priority factors, rationale, evidence, status	No recommendation bypasses action permissions.
Preference	Owner, key, value, scope, source type, confidence, state, expiry/decay	Explicit and learned preferences remain distinct.
Learning observation	Preference, signal type, source reference, timestamp, reversible status	Original observation is preserved; undo is non-destructive.
Job / notification	Owner, type, payload reference, status, attempts, schedule	Idempotent processing and bounded retries.
Audit event	Actor, event, resource, timestamp, request/job ID, metadata	No sensitive content or secrets in logs.
Export/deletion request	Owner, status, timestamps, artifact/reference	Lifecycle is observable and retryable.

7. API contract

Base path: **/api/v1**. Use JSON over HTTPS. All resource endpoints require a validated authenticated identity except explicitly public health/auth endpoints. Every operation **MUST** validate inputs and resource ownership, return consistent errors, support cursor pagination for collections, and include request/job identifiers where applicable. Retryable mutations **MUST** support idempotency keys.

Resource group	Endpoints / operations
Auth & account	POST /auth/register, /auth/login, /auth/logout, /auth/magic-link; GET /auth/providers, /me
Meetings	GET/POST /meetings; GET/PATCH/DELETE /meetings/{id}; participants; importance-assessment; prepare
Briefings	GET /meetings/{id}/briefings; latest; GET /briefings/{id}; evidence; feedback
Contacts & projects	CRUD /contacts and /projects; related commitments, milestones and progress
Commitments	CRUD /commitments; status and due-date updates with provenance
Preferences & learning	GET/PATCH /preferences; evidence; disable; POST /learning/observations/{id}/undo; review; history
Recommendations	GET /recommendations; status/dismissal and supported user actions
Follow-ups	Create/read/update draft messages and reminders; no send endpoint
Documents & outcomes	Create upload URL; register document; metadata/delete; meeting notes; outcome proposals; confirm/reject
Settings & privacy	Notification and retention settings; integrations list/revoke; exports; account deletion; audit events
Worker operations	Internal jobs: schedule prep, refresh briefing, review preferences, check documents, process outcomes/exports/deletion, send notification

API behavior requirements

- Use typed request/response schemas and generated OpenAPI documentation plus a developer guide and examples.
- Return stable machine-readable error codes with safe human-readable messages; do not leak cross-tenant resource existence.
- Enforce pagination, rate limits, payload/file-size limits and request timeouts.
- Worker-only routes must not be exposed as ordinary user capabilities; protect them with service authentication.
- Never log access tokens, provider keys, full sensitive prompts, transcripts or document contents.
- Do not provide any endpoint that sends client email/messages in the MVP.

8. Document ingestion and evidence

- Accept user-authorized uploads through scoped upload URLs; register metadata after upload.
- Validate file type, size and ownership; store objects privately and record checksums.
- Schedule document checks during preparation and before generation. Hash/deduplicate unchanged files to avoid needless reprocessing.
- Keep processing states and errors visible. Do not treat an unprocessed or inaccessible document as evidence.
- Cite evidence in the UI by source and location where available; retain the relationship between briefing claims and source records.
- Apply retention and deletion policies to originals, derived text, embeddings/memory references and exports.

9. Scheduling and job processing

- Persistent queue; jobs survive process restarts and are observable.
- Prioritize by meeting importance, deadline and user settings. Importance influences depth/timing, not permissions.
- Preparation schedule considers importance, complexity and available information.
- Significant changes trigger immediate refresh; minor changes are coalesced/queued.
- Use idempotent handlers, bounded retries with backoff, dead-letter/failure visibility and duplicate suppression.
- Worker rechecks user ownership and current authorization at execution time; queued payloads should reference records rather than embed sensitive content.
- Support cancellation/supersession of stale prep jobs and record the reason.

10. Security, privacy and account controls

- Supabase Auth with multiple authentication methods; validate issuer, audience, expiry and signature of tokens.
- Shared PostgreSQL with strict per-user authorization and RLS; test isolation across every resource family.
- Encrypt traffic in transit and data at rest, including backups. Keep secrets in hosting secret stores.
- Integrations, when added, must use least privilege and support revocation. Never request unnecessary scopes.
- Require clear authorization before importing or processing sensitive meeting content, including recordings/transcripts.
- Select and disclose cloud region during deployment.
- Provide configurable retention with a clear default selected before launch. Deletion removes data from active systems and expires backup copies according to the disclosed schedule.
- Export includes meeting records, preferences and learning history; protect generated exports and expire them according to policy.
- Expose security activity: sign-ins, integration changes, exports and consequential agent actions.
- Learning disablement does not erase existing preferences unless the user separately requests deletion.

11. UX requirements

- Personalized daily home dashboard with meetings, progress and recommendations.
- Concise briefing as default; expandable evidence and integrated conversational assistant.
- Structured screens plus chat; avoid forcing users into a chat-only interface.
- Global, meeting-type, project and contact preference scopes.
- Short onboarding followed by a guided first meeting.
- Manual refresh always available; show progress and meaningful failure/degraded states.
- Label assumptions, partial coverage, inferred outcomes and confidence. Ask only for material missing information.
- Post-meeting outcome updates are proposals; uncertain changes require confirmation.

12. LLM and agent safety

- Provider adapter abstracts Groq and a configurable fallback. Users do not select the model; the application chooses.
- If the primary LLM is unavailable, retry within bounded limits, use the configured fallback if available, and disclose degraded results.
- Constrain model tasks to structured outputs and domain operations; validate before persistence.
- Retrieved documents, meeting notes and memory are data, not instructions. Defend against prompt injection and cross-user retrieval.
- No autonomous external communication. No model-generated preference can alter permissions.
- Track model, prompt/version, token usage, latency, cost and outcome status without retaining sensitive content in ordinary logs.

13. Observability and operations

Initial operating assumptions and proposed targets:

Area	Initial requirement / target
Infrastructure budget	\$25–\$50 USD per month initially; alert before exceeding configured budget.
Backups	Daily database backups; verify restore procedure.
Recovery	Recovery time objective (RTO) ≤ 24 hours as a proposed target.
Availability	Proposed beta target: 99% monthly; not a guarantee.
API latency	Routine non-model API requests < 1 second p95 target; model-dependent operations excluded.
Monitoring	Application logs, uptime, job queue/worker health, model usage and cost alerts.
Health checks	/health/live, /health/ready, and restricted /health/dependencies.
Environments	Separate development, staging and production; no shared production secrets.
Release	Staged private prototype → beta → public release.

Deployment sequence: tests/build → backward-compatible database migrations → API → worker → frontend → smoke tests. Rollback procedures and migration compatibility must be documented.

14. Configuration and unresolved decisions

Use environment-specific configuration. The following names are illustrative and should be documented in .env.example without real secrets:

```
APP_ENV, APP_BASE_URL, API_BASE_URL, SUPABASE_URL, SUPABASE_ANON_KEY, SUPABASE_JWT_AUDIENCE, DATABASE_URL,
OBJECT_STORAGE_ENDPOINT, OBJECT_STORAGE_BUCKET, OBJECT_STORAGE_ACCESS_KEY, OBJECT_STORAGE_SECRET_KEY,
GROQ_API_KEY, GROQ_MODEL, FALLBACK_LLM_PROVIDER, FALLBACK_LLM_API_KEY, FALLBACK_LLM_MODEL, HINDSIGHT_URL,
```

```
HINDSIGHT_API_KEY, EMAIL_PROVIDER_API_KEY, EMAIL_FROM_ADDRESS, MONTHLY_LLM_BUDGET
```

Decisions to keep configurable or resolve during deployment: Render vs Railway; exact database and storage provider; fallback model/provider; email provider; retention defaults/periods; learning thresholds/weights/half-life; scheduler weights; cloud region. n8n is optional and MUST NOT be a core runtime or state dependency.

15. Repository structure

Suggested monorepo layout (the coding agent may refine names while preserving separation of concerns):

```
meeting-prep-agent/ apps/ web/ # Next.js + TypeScript UI, features, components, tests api/ # FastAPI app, routes, schemas, domain services app/ api/ core/ models/ schemas/ services/ agents/ providers/ repositories/ workers/ packages/ contracts/ # Shared API/domain contracts prompts/ # Versioned prompts and output schemas domain/ # Shared domain concepts supabase/ migrations/ seed.sql infra/ # Managed deployment config and scripts docs/ architecture/ api/ security/ operations/ adr/ .env.example README.md pyproject.toml package.json Makefile
```

16. Testing and acceptance criteria

- Functional tests cover meeting CRUD, importance override, preparation, refresh, recommendations, commitments, outcome confirmation, learning controls, export and deletion.
- Integration tests cover database, object storage, queue, LLM adapter, fallback and notifications using test doubles or isolated test services.
- Authorization tests prove users cannot read or mutate another user's meetings, contacts, projects, documents, briefings, preferences, exports or jobs.
- RLS tests validate tenant isolation at the database boundary, not only through API routes.
- Learning tests verify scope, contradiction handling, decay, disablement, evidence inspection and reversible undo.
- LLM tests validate schema conformance, partial briefing behavior, untrusted input handling, degraded mode and absence of send actions.
- Operational tests cover retries, idempotency, job recovery, backup/restore, retention, deletion and observability.
- UI tests cover onboarding, dashboard, briefing evidence expansion, chat, manual refresh, confirmation flows and privacy settings.

17. Delivery plan: vertical slices

Implement each slice end-to-end (UI, API, data and tests) rather than building all layers separately.

ID	Slice	Definition of done (summary)
MVP-001	Repository & CI foundation	Monorepo, formatting/lint/type checks, tests, build, environment template and contributor docs.
MVP-002	Auth & account isolation	Supabase Auth, account bootstrap, tenant context, RLS and cross-user denial tests.
MVP-003	Manual meetings & contacts	Create/edit/list meetings, contacts/projects, importance inputs and overrides.
MVP-004	Document ingestion	Private upload, metadata, validation, checksum/deduplication and processing status.
MVP-005	Briefing generation	Context retrieval, versioned briefing, evidence, partial/degraded states and manual refresh.
MVP-006	Scheduling & prioritization	Persistent jobs, importance/deadline priority, significant-change refresh and bounded retries.
MVP-007	Progress & recommendations	Commitments, milestones, blockers, progress and reasoned prioritized recommendations.
MVP-008	Behavioral learning	Preference states, scoped evidence, confidence/decay, disablement and undo.
MVP-009	Assistant & outcome capture	Integrated chat, notes/outcome proposals, confirmed updates and draft follow-ups only.
MVP-010	Privacy operations	Retention, export, deletion, integration revocation structure and audit visibility.
MVP-011	Observability & cost controls	Logs, health, job/model metrics, budget alerts and failure visibility.

ID	Slice	Definition of done (summary)
MVP-012	Release hardening	Security/integration tests, restore rehearsal, deployment runbook, staged release and smoke tests.

18. Definition of MVP complete

- A user can onboard, create a meeting manually, attach authorized documents and receive a scheduled or manually refreshed briefing.
- Briefings are concise, versioned, evidence-linked and honest about assumptions, missing data and degraded generation.
- The user can track progress and commitments, review recommendations, and confirm proposed outcomes.
- Learning is inspectable, scoped, reversible, confidence-aware and independently disableable; it never changes permissions.
- Exports, retention and account deletion follow documented policies; security-relevant events are visible.
- No client communication is sent by the agent; no UI/API/worker path bypasses user control.
- Automated tests demonstrate functional correctness, authorization, tenant isolation and operational recovery.
- Deployment, configuration, API documentation and incident/restore procedures are usable by another engineer.

19. Implementation notes for the coding agent

- Start with ADRs for tenancy/RLS, queue choice, storage, memory integration, provider abstraction and deletion semantics.
- Build a thin vertical slice early: auth → manual meeting → one briefing → evidence display → refresh.
- Keep domain logic in services, persistence behind repositories, and provider-specific code behind adapters.
- Use migrations as the schema source of truth; make migrations backward-compatible during staged deployment.
- Make job and API state transitions explicit and testable. Include correlation IDs across request, job, model call and audit event.
- Keep prompts versioned and outputs schema-validated. Store provenance and model metadata alongside generated content.
- Do not silently turn deferred integrations or optional n8n into MVP dependencies.
- When a requirement is ambiguous, preserve the user-control, privacy and traceability principles; record material assumptions in an ADR.

End of blueprint. This document consolidates the requirements and decisions recorded in the conversation; proposed targets and unresolved deployment choices are explicitly identified as such.

Integration Addendum: Hindsight & OpenClaw

This addendum extends the Meeting Prep Agent blueprint with deployment, configuration, API-boundary and integration requirements for Hindsight long-term memory and OpenClaw. Treat Hindsight and OpenClaw as independently versioned services. Pin versions and validate their exact API/CLI syntax against the release deployed; do not rely on unpinned latest tags in production.

A. Hindsight: deployment and configuration

A1. Role and topology

Hindsight is the contextual memory subsystem. PostgreSQL remains the authoritative source for meetings, contacts, projects, permissions, commitments, preferences, audit and workflow state. Hindsight stores/retrieves memory representations; it must never decide who can access a record or whether an action is permitted.

Recommended topology: deploy Hindsight as a private service reachable only from the FastAPI backend and worker. Keep it off the public internet. Use a separate Hindsight namespace/mental-model boundary per application user (or a rigorously isolated tenant boundary supported by the pinned version). Never put the Supabase service-role key or database credentials into Hindsight.

Setting	Recommended configuration / requirement
Service	Run the official Hindsight server using a pinned image/release. Keep the version in deployment config and update through staging first.
Network	Private service network; ingress limited to API and worker. TLS for traffic crossing a trust boundary.
Persistence	Use the storage backend and persistence mode supported by the selected Hindsight release; persist its data and include it in backup/restore planning.
Authentication	Enable the supported API authentication mechanism. Store the key/credential as a server-side secret; rotate it. If the deployment offers no built-in auth, enforce private networking and a trusted gateway.
Isolation	Derive memory scope from authenticated user ID on the server. Do not accept a caller-supplied tenant ID without checking it against the authenticated principal.
Retention	Define retention/deletion behavior for Hindsight memories, derived representations and indexes; make it consistent with the app's user retention and deletion policy.
Observability	Record request/job ID, operation, latency, status and usage metadata. Avoid logging raw memory text or sensitive meeting content.

A2. Data flow and memory lifecycle

- On an authorized user action or completed workflow, the backend decides whether a memory write is appropriate. Persist the canonical source record in PostgreSQL first; enqueue memory synchronization as a retryable job.
- Write only useful, permitted context (for example, durable user-confirmed facts, interaction summaries or outcomes). Exclude sensitive meeting content from behavioral preference learning. Do not treat every transcript sentence as a durable memory.
- Attach provenance metadata: user/tenant scope, source entity ID, source type, source version/checksum, created time, sensitivity classification and deletion reference, to the extent supported by the deployed API.
- On briefing preparation, retrieve memory only after authenticating the user and checking the meeting/project/contact ownership in PostgreSQL. Apply relevance and recency limits; combine with authoritative structured records.
- Treat retrieved memory as untrusted reference material. It may inform the answer but cannot override system policy, permissions, explicit instructions or source-of-truth records.
- When source records are corrected or deleted, enqueue corresponding memory correction/deletion. Make synchronization idempotent and observable; provide a reconciliation job for drift.

A3. Backend API adapter

Create a HindsightAdapter in the FastAPI provider layer. Keep Hindsight-specific URLs, payloads and SDK calls inside this adapter; domain services should call application-level methods such as remember(), recall(), reflect() (only if supported and needed), and forget(). Confirm exact endpoint names and schemas against the pinned Hindsight OpenAPI/SDK rather than hard-coding guessed routes.

Suggested internal interface (illustrative application contract, not Hindsight's wire API):

```
class MemoryAdapter(Protocol):
    async def remember(self, *, user_scope: str, source_id: str, source_version: str, content: str, metadata: dict) -> MemoryWriteResult: ...
    async def recall(self, *, user_scope: str, query: str, filters: dict, limit: int) -> list[MemoryHit]: ...
    async def forget(self, *, user_scope: str, source_id: str) -> ForgetResult: ...
    async def health(self) -> DependencyHealth: ...
```

The adapter must: apply timeouts; retry only safe/idempotent operations; normalize vendor errors; validate response schemas; enforce maximum result count/token budget; attach correlation IDs; redact sensitive values from logs; and expose metrics. Use a circuit breaker or equivalent failure isolation so memory outages do not take down manual meeting CRUD.

A4. Suggested configuration

Keep secrets in the host's secret manager. Example names are application-level conventions; map them to the actual Hindsight deployment:

```
HINDSIGHT_BASE_URL=http://hindsight.internal: HINDSIGHT_API_KEY= HINDSIGHT_TIMEOUT_SECONDS=10
HINDSIGHT_MAX_RECALL_ITEMS=8 HINDSIGHT_MAX_CONTEXT_TOKENS= HINDSIGHT_ENABLED=true HINDSIGHT_SYNC_QUEUE=
```

Do not expose these values to the browser. The port, auth header, database variables and available Hindsight operations must be taken from the chosen release's official deployment documentation.

A5. Failure behavior and acceptance tests

- If Hindsight is unavailable, continue with PostgreSQL-backed context and mark the briefing as memory-degraded when relevant. Do not fabricate recalled facts.
- Queue writes for bounded retry; deduplicate by source ID/version or an equivalent idempotency key. Surface persistent failures to operators.
- Verify isolation with two test users: a recall for user A must never return user B's content, even if a malicious filter or source ID is supplied.
- Verify correction, deletion, retention expiry, restart persistence, timeout handling and schema-validation failures.
- Measure retrieval latency and result size; enforce a context budget so memory cannot crowd out critical current meeting facts.

B. OpenClaw integration

B1. Role and rollout

OpenClaw is a future integration/agent-execution layer for meeting-related systems, conditional on supported connectors, platform terms and user-granted permissions. It is not the product's source of truth and is not required for the manual-first MVP. Keep it behind a feature flag and an IntegrationAdapter so the core product remains usable when it is absent.

Do not give OpenClaw unrestricted access to the app database, Supabase service-role credentials, Hindsight administration or arbitrary shell/browser actions. The FastAPI service should expose narrow, typed, user-scoped capabilities to an OpenClaw workflow, or consume narrowly scoped events from OpenClaw. Prefer an explicit allowlist of tools.

B2. Trust and authorization model

- Each integration connection belongs to one app user and records provider, granted scopes, token reference, status, creation/expiry and revocation time. Encrypt refresh/access tokens at rest; never return them to the browser.
- OAuth authorization, if supported by the connector, must use state/PKCE where applicable, exact redirect URI allowlists, least-privilege scopes and explicit consent. If the integration uses another credential model, document its security properties.
- OpenClaw tool calls must carry a verifiable user context established by the backend, not a user ID supplied by model-generated arguments.
- Authorize every operation at execution time, including background jobs. Re-check connection status and scopes before using credentials.
- MVP allowlist: read authorized meeting/context data only where explicitly enabled; create a draft/reminder only when requested and supported. Block send, delete, invite, purchase, payment and other consequential actions.
- No agent-generated message may be sent to a client. Draft creation is distinct from sending and must remain visibly labeled as a draft.

- All external content returned by a connector is untrusted input and must be isolated from instructions. Sanitize, bound and provenance-tag imported content.

B3. Integration architecture

Recommended flow: user grants a connection in the web UI → FastAPI records consent and stores credentials securely → a scoped integration job invokes the OpenClaw integration adapter → adapter validates user, connection, scope and allowed operation → connector action executes → normalized result is validated and persisted by FastAPI → audit event and user-visible status are recorded. If OpenClaw initiates a callback, validate its signature/authentication, timestamp and replay protection before processing.

Keep connector-specific logic behind an interface such as:

```
class IntegrationAdapter(Protocol):
    async def list_capabilities(self) -> list[Capability]: ...
    async def execute(self, *, user_id: UUID, connection_id: UUID, operation: AllowedOperation, arguments: dict, idempotency_key: str) -> IntegrationResult: ...
    async def revoke(self, *, connection_id: UUID) -> None: ...
```

This is the application's internal interface, not a claim about OpenClaw's native API. Use the official OpenClaw documentation for the exact gateway, authentication, configuration and tool-registration syntax of the pinned release.

B4. Configuration and deployment

Deploy OpenClaw as a separate, private service only when a supported integration is implemented. Pin its version, isolate its runtime, restrict outbound network access where practical, and provide a separate service credential for the app-to-OpenClaw boundary. Do not assume a generic OpenClaw install is safe for multi-tenant use without verifying its isolation model.

```
OPENCLAW_ENABLED=false OPENCLAW_BASE_URL=http://openclaw.internal: OPENCLAW_SERVICE_TOKEN=
OPENCLAW_TIMEOUT_SECONDS=15 OPENCLAW_ALLOWED_OPERATIONS= OPENCLAW_WEBHOOK_SECRET=
```

These are proposed application configuration names. Confirm whether the selected OpenClaw release supports the required auth, tenancy, tool restrictions and webhook verification. If it does not, enforce those controls in a trusted gateway or defer that integration.

B5. Revocation, audit and failure behavior

- Revocation must disable the connection immediately in the app, stop queued actions, invalidate/delete stored credentials where possible, and invoke provider-side revocation when supported.
- Log connection creation, scope changes, tool/action name, actor, target resource, outcome and correlation ID. Do not log credentials or sensitive payloads.
- Use idempotency keys for retryable operations. Do not automatically retry non-idempotent actions unless the provider supports safe deduplication.
- On timeout or uncertain result, mark the action outcome unknown and reconcile before retrying; never claim an action succeeded without confirmation.
- Provide user-visible connection status, last successful sync/action, granted scopes and revoke control.

C. Integration sequence and implementation gates

Phase	Work	Exit criteria
MVP foundation	Implement adapter interfaces, feature flags, secret handling, audit schema and mocked tests. Keep both external services optional in local development.	Core app works with both integrations disabled; no provider secrets in client bundle.
Hindsight pilot	Deploy private Hindsight; wire health check, scoped remember/recall, memory-sync queue and deletion path.	Cross-user isolation, retry/idempotency, restore and deletion tests pass.
OpenClaw evaluation	Pin a version; verify supported connectors, auth, tool controls, isolation, webhook behavior and licensing/platform terms.	Documented threat model and an approved allowlist; unsupported capabilities remain disabled.
OpenClaw pilot	Enable one narrowly scoped connector behind a feature flag; test consent, execution, revocation and audit end to end.	No send path; user context is server-derived; failures are visible and recoverable.
Production rollout	Deploy to staging, run security tests, then staged beta rollout with monitoring and rollback.	Operational owner, runbook, budget alerts and incident/revocation procedure in place.

D. Security and integration test checklist

- Tenant isolation: memory retrieval and connector actions cannot cross user boundaries.
- Prompt injection: malicious text in a document, memory or external connector response cannot authorize tools or alter policy.
- Least privilege: verify each connector scope and operation against the approved allowlist.
- Secret hygiene: scan logs, traces, errors, frontend bundles and exported diagnostics for tokens and sensitive content.
- Revocation: prove queued and new actions are denied after revocation; verify provider revocation where supported.
- Resilience: test timeouts, retries, duplicate callbacks, replay attempts, malformed payloads, service restart and dependency outage.
- Data lifecycle: prove retention/deletion applies to PostgreSQL, object storage, Hindsight and connector-held data to the extent the provider supports it.
- Human control: verify no client-send operation exists in UI, API, worker tool registry or OpenClaw allowlist.

E. Documentation the coding agent must produce

- Hindsight deployment guide: pinned version, environment variables, auth/network setup, persistence, backup/restore, upgrade and deletion behavior.
- Hindsight API/SDK mapping: exact endpoints/methods and schemas for the pinned release, error mapping, timeouts and idempotency strategy.
- OpenClaw integration guide: supported version/connectors, architecture diagram, credential flow, approved scopes/tools, configuration and revocation procedure.
- Threat model and data-flow diagram showing trust boundaries, tenant context and where sensitive data is stored or transmitted.
- Runbooks for Hindsight outage, OpenClaw outage, credential compromise, integration revocation and failed deletion.

Implementation caution: Hindsight and OpenClaw change independently. The coding agent must verify native API names, configuration keys, authentication headers, supported storage and connector capabilities against the exact versions selected. This blueprint specifies the application integration contract and safety requirements; it intentionally does not invent vendor-specific wire endpoints.

Vendor setup supplement: concrete configuration patterns

This supplement reflects the vendor documentation checked for this revision. Pin the exact Hindsight and OpenClaw versions used; their APIs and configuration can change. These instructions complement—not replace—the application-level isolation and human-approval requirements.

1. Hindsight setup

1.1 Install and connect

Use the official Hindsight server deployment instructions for the selected release. For local development, start with its documented local server and Python client; install the client package `hindsight-client`. Keep the server private in staging/production. The API service and worker should be the only application components that can call it.

```
# Python dependency (pin a tested version in the lockfile) pip install hindsight-client # Application-side
client construction (illustrative) from hindsight_client import Hindsight client = Hindsight(
base_url=settings.hindsight_base_url, api_key=settings.hindsight_api_key, # when API-key auth is enabled )
```

Hindsight exposes three core operations: **retain** (ingest and extract memories), **recall** (retrieve relevant memories), and **reflect** (generate a response using memory and disposition). Prefer the official SDK for the pinned release; use the official OpenAPI/API reference if calling REST directly. Do not assume Cloud and self-hosted URL prefixes or authentication are interchangeable.

1.2 Memory banks and application mapping

Create a separate memory bank for each application user, using an opaque server-generated bank ID. Persist the mapping `(app_user_id → hindsight_bank_id)` in PostgreSQL with a unique constraint. Never let the client choose the bank ID. If the deployed edition offers a stronger tenant-isolation primitive, document and test it before relying on it. Do not put multiple users into one shared bank.

Use the app's source IDs as stable document identifiers when supported. Supply a meaningful context label (for example, `'meeting outcome'`, `'contact update'`, `'project status'`) and the event timestamp. Hindsight's retain documentation notes that context influences extraction and that re-retaining an updated conversation with the same document ID replaces/reprocesses that document; therefore use stable IDs only when replacement semantics are intended.

1.3 Suggested Hindsight tuning

Start with conservative ingestion: retain confirmed outcomes, durable user-approved facts and concise summaries rather than every raw interaction. Use a clear retain mission to focus extraction on professional context, commitments, preferences explicitly approved for memory, and project facts. Keep behavioral preference learning in the application's own tables; do not let memory extraction silently create permanent preferences.

Hindsight's configuration is split by service: `HINDSIGHT_API_*` for the memory API and `HINDSIGHT_CP_*` for the control plane. Configure only the services you deploy. The API supports per-operation LLM settings for retain and reflect; recall itself is retrieval and does not require an LLM. If using Groq, verify the selected model supports the structured output required by retain. Set explicit timeouts, bounded retries and concurrency appropriate to the small initial budget.

```
# Illustrative server-side settings; confirm defaults for pinned release HINDSIGHT_API_LLM_PROVIDER=groq
HINDSIGHT_API_LLM_API_KEY= HINDSIGHT_API_LLM_MODEL= HINDSIGHT_API_RETAIN_MISSION=Focus on durable, factual
professional context, confirmed commitments, project facts and meeting outcomes. Exclude secrets and sensitive
personal details. HINDSIGHT_API_RETAIN_LLM_MAX_RETRIES=3 HINDSIGHT_API_RETAIN_LLM_TIMEOUT=
HINDSIGHT_API_RECALL_MAX_TOKENS=
```

Use separate retain/reflect model configuration only after measuring quality and cost. Configure operation retention and data deletion deliberately; operation-log retention is not the same as memory retention. Validate backup/restore and deletion across the memory store.

1.4 Application API usage

Implement the adapter with the official client methods, mapping app operations to `client.retain(bank_id=..., content=..., context=..., timestamp=...)`, `client.recall(bank_id=..., query=...)`, and `client.reflect(bank_id=..., query=...)` where appropriate. Prefer recall for retrieval that will be composed by the Meeting Prep Agent; use reflect only when Hindsight's own generated response is explicitly wanted. Validate result shape and provenance before showing or persisting results.

Do not send all meeting content to Hindsight by default. Apply the application's consent and sensitivity rules before each retain. For corrections and deletion, use the supported document/memory deletion operations for the pinned release and maintain a reconciliation job for failures.

2. OpenClaw setup

2.1 Gateway installation and safe baseline

OpenClaw is a Gateway with a broad control surface. Install a pinned release using the official onboarding flow (``openclaw onboard``), then run ``openclaw doctor`` and ``openclaw models status``. Its default local Gateway port is commonly 18789; configure the actual port explicitly. For a local-only pilot, bind to loopback. For a server deployment, use private ingress and authentication; never expose the Gateway directly to the public internet.

```
{ gateway: { mode: "local", port: 18789, bind: "loopback", auth: { mode: "token" } } }
```

Store the Gateway token on the host as a secret (the documented environment variable is ``OPENCLAW_GATEWAY_TOKEN``). Restart the service after changing environment/configuration and validate with ``openclaw doctor``. Do not place the token in browser code, prompts, logs or user-visible settings.

2.2 Critical API security distinction

The documented HTTP ``POST /tools/invoke`` endpoint uses Gateway authentication and tool policy. With shared-token/password authentication, possession of the bearer credential is treated as trusted operator access with broad authority; a narrower ``x-openclaw-scopes`` header does not reduce that authority. Therefore, **do not call ``/tools/invoke`` from a multi-tenant user request path using a shared Gateway token**. Treat that token as an operator secret, not a per-user integration credential.

For the Meeting Prep Agent, prefer an app-owned allowlisted connector adapter that calls only specific provider APIs with per-user credentials. If OpenClaw is used, isolate it as a single-purpose service/agent with private ingress, narrowly defined tools and no broad access to user data. Where a gateway tool invocation is unavoidable, place a trusted authorization proxy in front of it and test the exact identity/scope behavior of the pinned OpenClaw version. Do not assume scopes are effective with shared-token mode.

2.3 Rollout and permissions

Begin with OpenClaw disabled in production. Pilot one read-only capability at a time. Require explicit user consent and least-privilege connector scopes. Maintain a server-side allowlist of tool names and arguments. Block send, delete, invite and other consequential actions in the app adapter, tool registry and connector configuration—not merely in the prompt. Recheck connection state and authorization immediately before execution.

If OpenClaw returns a timeout or ambiguous result, mark the operation ``unknown`` and reconcile before retrying. Log action metadata and outcome, not message bodies or credentials. Revocation must stop queued work, disable the connection and revoke/delete provider credentials where supported.

3. Coding-agent implementation gate

Before implementing either integration, have the coding agent record: (1) exact pinned release and official docs URL; (2) deployed service topology; (3) API/SDK calls and request/response schemas; (4) auth mechanism and secret rotation; (5) tenant-isolation proof; (6) retention/deletion behavior; (7) operational health checks; and (8) security tests. If the selected OpenClaw deployment cannot provide enforceable isolation and narrow authorization, keep it out of the production request path.

Vendor documentation used for this supplement: Hindsight developer API quickstart and configuration reference; OpenClaw Gateway configuration and HTTP tools-invoke documentation. Verify current vendor documentation during implementation.